FORGE

SPRINT TASKS PLAN

Detailed implementation checklist for the project team

Based on the uploaded FORGE EPIC Breakdown, revised User Stories, and corrected FR/NFR document.

Planning assumption: Six sequential sprints are proposed as a practical starting plan. Sprint duration is not specified in the uploaded material; use your course/team cadence and capacity to size each sprint. This document contains sprint tasks only, plus the consistency checks needed before committing conflicting stories.

Team rule: complete, test and integrate a small working increment every sprint. Do not treat an EPIC as automatically equal to one Sprint.

Sprint plan at a glance

The EPIC document correctly groups 58 user stories into 13 EPICs and states that stories should be scheduled individually according to priority, dependency, estimate and capacity. The sequence below follows those dependencies: foundation → collaboration → workflow/feed → GitHub/insight → discovery/social → AI/moderation/release.

Sprint	Goal	Stories
Sprint 1	Foundation: account, profile, project and permissions	US-01, US-02, US-03, US-05, US-08, US-09
Sprint 2	Core collaboration: membership, channels, tasks and Kanban	US-16, US-18, US-21, US-22, US-24, US-25, US-26, US-27, US-28
Sprint 3	Project workflow: invitations, decisions, feed and notifications	US-10, US-11, US-12, US-13, US-14, US-17, US-20, US-23, US-29, US-30, US-35, US-36
Sprint 4	Project knowledge, files, dashboard and GitHub	US-15, US-31, US-32, US-33, US-34, US-37, US-38, US-39, US-40
Sprint 5	Discovery, profiles, social feed and engagement	US-06, US-07, US-45, US-46, US-47, US-48, US-49, US-50, US-51, US-52, US-53, US-54
Sprint 6	AI copilot, moderation, security hardening and release	US-04, US-19, US-41, US-42, US-43, US-44, US-55, US-56, US-57, US-58

Before Sprint 1: assign a story owner, estimate effort, confirm dependencies, and put the stories into a sprint backlog. The story IDs listed here are a proposed sequencing, not a change to the source priorities.

Sprint 1: Foundation: account, profile, project and permissions

Sprint goal: Establish a working, secure foundation so users can create accounts, manage profiles, create projects and enforce project roles.

Stories in scope: US-01, US-02, US-03, US-05, US-08, US-09

Dependencies: None; this sprint creates the foundation. Resolve the authentication token specification conflict first.

A. Team alignment and repo setup

- Confirm the corrected FR/NFR document is the source of truth and resolve the conflicts listed at the end before coding affected stories.
- Create the repository structure, protected main branch, feature-branch naming, pull-request template, issue labels and sprint board columns: Backlog, Ready, In Progress, Review, Test, Done.
- Agree the stack, environment-variable naming, API error format, lint/format rules and module boundaries. Keep secrets out of Git.
- Set up local development instructions, .env.example, basic CI and a repeatable database setup/seed process.

B. Data model and backend foundation

- Design and document User, Profile/SkillTag, Project, ProjectMembership/Role and Session/RefreshToken models.
- Implement database connection, centralized error handling, request validation, authentication middleware and a consistent JSON response convention.
- Create initial API route skeletons and endpoint documentation for registration, login, logout, profile and projects.
- Add server-side authorization helpers. Enforce role permissions on the server; hiding a button in the UI is not authorization.

C. Authentication — US-01, US-02, US-03

- Build sign-up form and API: validate email, enforce unique account, require password of at least 8 characters, store only a salted password hash.
- Build login API and protected-route handling. Implement the agreed token design only after resolving 24-hour JWT versus 15-minute access + 7-day refresh-token conflict.
- Implement logout so the refresh/session token is revoked and later protected requests using the logged-out session are denied.
- Handle duplicate email, invalid credentials, expired/revoked sessions, server errors and loading/success/error states.

D. Profile — US-05

- Build profile edit page for photo, bio and skill tags.
- Validate photo type JPEG/PNG and maximum 2 MB; validate bio maximum 500 characters and no more than 10 skill tags.
- Keep the previous photo if a replacement fails validation. Store uploads in persistent external storage, not hosting local disk.
- Add server-side tests for valid updates, invalid file type/size, overlong bio, tag limit and unauthorized profile edits.

E. Project creation and roles — US-08, US-09

- Build create-project form with name (required, max 80 chars), description, domain from an agreed fixed list, Public/Private visibility and optional starting members/roles.
- On creation, set the creator as Owner and create membership records for selected starting members.
- Implement Owner/Admin/Contributor/Viewer role hierarchy and permission checks from the role-permission matrix.
- Test denied actions with HTTP 403, especially Contributor/Viewer role changes and Admin attempts to change/grant Owner.

Sprint review / exit checks

- A new user can register, log in, edit profile, create a project and access only permitted project actions.
- Automated tests cover core success and failure paths; CI passes; setup documentation works on a clean clone.
- Demo the complete path with two test accounts and at least one project with different member roles.

Sprint 2: Core collaboration: membership, channels, tasks and Kanban

Sprint goal: Deliver the first useful project workspace: teammates can communicate, create/assign/update tasks and track work on a Kanban board.

Stories in scope: US-16, US-18, US-21, US-22, US-24, US-25, US-26, US-27, US-28

Dependencies: Sprint 1 user/project/membership models, role checks, database and CI.

A. Project workspace and membership foundation

- Build the project home/workspace layout and project member list using the Sprint 1 models.
- Ensure every private-project endpoint checks membership and role on the server.
- Add project-level navigation for channels, tasks and milestones.

B. Real-time project channels and mentions — US-16, US-18

- Create the default #general channel when a project is created, or define a deterministic channel-creation flow.
- Implement message persistence, chronological history, sender identity and timestamps.
- Add real-time delivery (WebSocket or the chosen equivalent) and reconnect/history behaviour; target delivery under 1 second under classroom-scale use.
- Implement @mention suggestions limited to current project members and create a mention event/notification record for the named member.
- Ensure Viewers can read channels but cannot send messages; reject unauthorized API requests with 403.

C. Tasks and assignment — US-21, US-22, US-24

- Create Task model with project, title, optional description, label, assignee, status, due date, milestone link and version/update metadata as agreed.
- Build task creation with required title and initial status To Do; validate assignee belongs to the project.
- Implement assignment and generate an assignment event for later notifications.
- Implement status updates and publish a project activity event when status changes.
- Resolve whether Blocked is a status or a label before finalizing the model; corrected FR-7b says status Blocked while US-40 refers to a Blocked label.

D. Milestones — US-25, US-26

- Build milestone creation for Owner/Admin only, with title and optional target date.
- Allow authorized Contributors to attach tasks to a milestone.
- Display progress as completed tasks / total attached tasks; handle zero-task milestones without division errors.
- Apply the agreed priority after reconciling US-25/US-26 with corrected FR-7c.

E. Kanban board — US-27, US-28

- Build To Do, In Progress and Done columns; include Blocked if the team confirms it is a task status.
- Render each task in the column matching its persisted status.
- Implement drag-and-drop status updates and optimistic UI if supported; update UI within 300 ms of drop.
- If the API rejects an update, return the card to its prior column and show a clear error.
- Add conflict detection/version checks so a stale update does not silently overwrite a newer task change.

Sprint review / exit checks

- Demo two project members collaborating: one creates and assigns a task, another sees the task and status update.
- Verify role restrictions for Viewer and Contributor/Admin on task and milestone actions.
- Test drag success, API failure rollback and concurrent task update conflict.

Sprint 3: Project workflow: invitations, decisions, feed and notifications

Sprint goal: Make team coordination dependable through membership workflows, decisions, a shared activity feed and in-app notifications.

Stories in scope: US-10, US-11, US-12, US-13, US-14, US-17, US-20, US-23, US-29, US-30, US-35, US-36

Dependencies: Sprint 2 task/channel models. Build a shared event schema early so feed and notifications consume the same events.

A. Membership workflow — US-10 to US-14

- Implement invite-existing-user flow for Owner/Admin, including chosen role, pending state and duplicate/member checks.
- Implement accept/decline invitation. Invitations expire after 7 days; accepting creates membership with the invited role.
- Implement public-project join-request queue for managers; approve or reject. A rejected user cannot re-request for 7 days.
- Implement member removal while preserving historical messages/tasks. Prevent removing the only Owner.
- Implement Leave Project with confirmation; prevent the only Owner from leaving until ownership is transferred.
- Test authorization and state transitions for pending, accepted, declined, rejected and expired requests.

B. Thread replies and task comments — US-17, US-20

- Add threaded replies to a channel message with reply count if retained in scope; corrected FR-5 calls threads a stretch goal.
- Add task comments with author and timestamp; support @mentions in comments if the team commits to the related notification behaviour.
- Do not let optional work block Must-level messaging, tasks or permissions.

C. Due dates and deadline events — US-23

- Add due-date editing and display on task cards/details.
- Make due dates available to upcoming-deadline queries; deadline notifications apply when a task is not Done and due in less than 24 hours.
- Test timezone/date-boundary cases and ensure Done tasks do not trigger deadline notifications.

D. Decision records — US-35, US-36

- Build decision creation with title, description, at least one project-member participant, date and optional related task/thread link.
- Validate required fields server-side; invalid submissions must not persist.
- Build decision list and edit flow, respecting project roles; links must navigate to the referenced task or thread.
- Confirm story priorities against corrected FR-13/FR-34 before sprint commitment.

E. Shared project Activity Feed — US-29

- Create a shared event schema with event type, actor, project, timestamp, source-object ID and safe display metadata.
- Emit events for channel messages, task status changes and other implemented activity. Reserve a normalized event type for GitHub events in Sprint 4.
- Render a newest-first feed showing event type, actor and time; paginate or limit results to avoid unbounded loads.
- Enforce private-project membership checks and target load time under 3 seconds for up to 100 concurrent viewers.

F. In-app notifications — US-30

- Create notification storage and UI/list with type, timestamp, read/unread state and a link to the relevant message/task.
- Generate mention, assignment and deadline notifications; target appearance within 5 seconds of the event.
- Opening a notification navigates to the related item and marks it read.
- Use clear empty, loading, success and error states; keep notification events idempotent where retries are possible.

Sprint review / exit checks

- Demo invitation/join request, decision editing, chronological feed, and mention/assignment/deadline notifications.

- Check feed privacy, role permissions, duplicate invitation/request handling and the 5-second notification target.
- Record any optional story not completed and return it to the backlog rather than silently dropping it.

Sprint 4: Project knowledge, files, dashboard and GitHub

Sprint goal: Make project work inspectable and connected to its engineering work through files/docs, dashboard/health and GitHub events.

Stories in scope: US-15, US-31, US-32, US-33, US-34, US-37, US-38, US-39, US-40

Dependencies: Sprint 1–3 project, task, feed and membership capabilities. GitHub credentials/webhook secrets must be configured securely.

A. Project completion — US-15

- Allow the Owner to mark an active project Completed and reopen it to Active.
- Confirm whether Archived from corrected FR-3 is in scope; do not implement a new status without agreeing its behaviour and permissions.
- Ensure completion state is available for later portfolio queries.

B. File uploads and documentation — US-37, US-38

- Implement project file upload/list/download with persistent external storage.
- Enforce the proposed 10 MB maximum server-side and show a clear rejection message above the limit.
- Implement Markdown documentation create/save/list/read with maximum 10,000 characters if retaining US-38.
- Sanitize or safely render Markdown to prevent unsafe HTML/script injection; restrict write operations to authorized project roles.
- US-38 is Should in the user-story file but corrected FR-14b is Could; confirm scope before committing effort.

C. Dashboard and project health — US-39, US-40

- Build dashboard showing milestone progress, count of tasks not Done and tasks due in the next 7 days.
- Build project-health view listing overdue tasks (past due and not Done), blocked tasks and activity level over the last 7 days.
- Use agreed activity thresholds: Low <5 events, Medium 5–20, High >20 in 7 days.
- Ensure dashboard queries use only the current project's data and are permission-protected.

D. GitHub OAuth and repository — US-31, US-32, US-33

- Register/configure the GitHub OAuth app and keep client secrets only in server environment variables.
- Implement Owner authorization and safe callback/state validation. If access is denied, do not persist a connected state.
- Allow the Owner to select one repository and show its connection as Active.
- Allow permitted Owner/Admin to view status, disconnect and re-authorize; after disconnect, stop ingesting new events but retain historical feed entries.
- Encrypt GitHub access tokens at rest and never return them to the browser or log them.

E. GitHub events — US-34

- Configure signed webhooks for commits, issues and pull requests. Verify X-Hub-Signature-256 before processing.
- Make webhook processing idempotent to avoid duplicate feed entries and record enough delivery metadata for troubleshooting.
- Map each accepted event to the shared Activity Feed schema and include type, title, author and GitHub link.
- Target feed appearance within 60 seconds; respect GitHub API rate limits and document retry/logging behaviour.

Sprint review / exit checks

- Demo dashboard/health views and a project with files/docs if in scope.
- Connect a test repository, trigger a commit/issue/PR event and show it in the project feed.
- Verify invalid webhook signature rejection, disconnect behaviour, and that core chat/tasks/feed continue if GitHub is unavailable.

Sprint 5: Discovery, profiles, social feed and engagement

Sprint goal: Deliver public discovery and community engagement while protecting private project and profile information.

Stories in scope: US-06, US-07, US-45, US-46, US-47, US-48, US-49, US-50, US-51, US-52, US-53, US-54

Dependencies: Profile/project models, completed-project/task/decision/message data, membership/privacy rules and image storage.

A. Public profiles and privacy — US-06, US-07

- Build read-only view of another user's photo, bio, skill tags, follower count and agreed public score statistic.
- Implement privacy preferences for collaborator-search visibility and public portfolio visibility.
- Do not expose fields a user has hidden; a disabled public portfolio link must show a not-available response.

B. Project discovery and joining — US-45, US-46, US-47

- Build public-project browse page with domain, skill-tag and status filters plus keyword search over name/description.
- Never list private projects in visitor discovery results.
- Build public read-only project page showing only name, description, domain, status and member names; hide channels, messages, tasks and files from non-members.
- Allow authenticated non-members to submit one pending join request with optional message; visitors must be asked to log in.
- Integrate manager approval/rejection from Sprint 3.

C. Collaborator search and portfolio — US-48, US-49

- Implement skill-tag search with case-insensitive matching; return users matching at least one selected tag, ordered by number of matching tags.
- Exclude users who opted out of collaborator search.
- Build portfolio from completed projects, roles and per-project contribution stats: tasks completed, decisions recorded and messages posted.
- Keep portfolio statistics no more than 24 hours old and respect public-portfolio privacy settings.
- Confirm US-49 Should versus corrected FR-21 Could before final scope commitment.

D. Public posts — US-50, US-51

- Build project-independent public post creation with text up to 500 characters and optional one JPEG/PNG/WebP image up to 5 MB.
- Store images in persistent external storage, validate type/size on the server, and provide visible success/error feedback.
- Apply per-user rate limit of 10 posts per hour.
- Allow authors to edit their own post (mark as edited) or delete after confirmation; reject attempts to edit/delete another user's post with 403.

E. Voting, trending and following — US-52, US-53, US-54

- Implement one active upvote/downvote per user per post, with atomic handling so concurrent requests do not double-count.
- Align behaviour with corrected FR-24: users may change or retract their vote. Prevent repeat same-direction voting from changing score.
- Implement trending score = (upvotes - downvotes) / (age_hours + 2)^1.5; recalculate at least every 5 minutes and break ties by newer post first.
- Target public-feed load under 3 seconds under classroom-scale use; rate-limit voting to 30 votes per minute per user.
- Implement follow/unfollow and public follower counts. Align profile statistic terminology with corrected FR-26 (total vote score = sum of net scores), rather than mixing 'likes' and 'votes'.

Sprint review / exit checks

- Demo public project discovery, privacy-safe profiles, a public post with an image, voting, trending order and follow/unfollow.

- Test private-project exclusion, privacy opt-outs, one-vote integrity under concurrent requests, rate limits and invalid image rejection.
- Check mobile-sized and laptop-sized browser layouts.

Sprint 6: AI copilot, moderation, security hardening and release

Sprint goal: Complete advanced AI and moderation capabilities, resolve cross-cutting quality risks, and prepare a stable demonstrable release.

Stories in scope: US-04, US-19, US-41, US-42, US-43, US-44, US-55, US-56, US-57, US-58

Dependencies: Permission-filtered project data, activity history, notifications, public-post/report models and deployment environment.

A. Password reset and direct messages — US-04, US-19

- Implement password reset with a single-use link valid for 1 hour; use the same confirmation response for registered and unregistered email addresses.
- On successful reset, invalidate existing sessions. Configure and test an email provider or document the deployment dependency.
- Implement direct conversations separate from project channels; only the two participants can read messages.
- Target direct-message delivery under 1 second under classroom-scale use. Confirm US-19 Must versus corrected FR-6 Should.

B. AI Copilot and Catch Me Up — US-41 to US-44

- Implement project-member question submission up to 500 characters and show a loading indicator after 2 seconds.
- Collect only project messages, tasks, decisions and GitHub events that the requesting member is authorized to access; enforce permission filtering before sending context to the LLM.
- Keep each request within the agreed/documented 8,000-token input budget; trim context deterministically and document what is omitted.
- Return answers with links/sources to project records used. If data cannot answer, explicitly say there is insufficient project data and do not invent project-specific claims.
- Implement Catch Me Up using activity since the user's previous successful login; if nothing changed, say so.
- Handle LLM provider outage with an AI-unavailable message while chat, tasks and feeds continue to work. The Copilot must not create or modify project data autonomously.

C. Moderation and platform administration — US-55 to US-58

- Implement report-post flow with a defined reason and duplicate-report prevention.
- Create moderator queue and implement the agreed actions: remove post, warn user via in-app notification, or dismiss report.
- Implement Platform Admin assignment/revocation of moderator role; bootstrap the first Platform Admin using a seed script or environment configuration.
- Implement account suspension/reactivation and reject protected actions by suspended accounts according to agreed rules.
- Audit all moderator/admin routes for server-side authorization and log moderation outcomes without exposing secrets.

D. Quality, security, integration and release

- Run end-to-end tests across register/login/logout, role permissions, project privacy, tasks/Kanban, notifications, GitHub events, public feed, voting and moderation.
- Test NFRs: salted password hashes; short-lived/revocable sessions per agreed auth design; HTTPS in deployment; input validation; role-protected APIs; loading/error feedback; responsive layout.
- Add basic automated tests for every backend route and ensure CI tests and coding-convention checks pass before merge.
- Document API endpoints, request/response behaviour, setup steps, environment variables, deployment steps, known limitations and demo accounts.
- Deploy web app/backend/database/storage; verify cold-start/loading messaging and availability during the demo window.
- Run a final data/privacy review: private project data never appears in public pages, unauthorized API calls fail, GitHub secrets/tokens are not exposed, and AI context is permission-filtered.

Sprint review / exit checks

- Demonstrate one end-to-end user journey across project collaboration, AI summary, public social feed and moderation.
- All Must stories selected for the release pass their acceptance criteria; unresolved items are explicitly listed as limitations/backlog.

- CI is green, deployment is reachable, seed/admin setup is documented, and each team member's work is traceable through commits and pull requests.

Cross-sprint working rules

Use these checks in every sprint, not only at the end.

Definition of Ready (before a story enters a sprint)

- Story ID, user-visible outcome and acceptance criteria are understood by the team.
- Dependencies and role/permission implications are known; external services and credentials are identified.
- Priority conflicts below are resolved for any story the team plans to implement.
- A developer and reviewer are assigned; test cases and expected outcomes are written before implementation.
- The story is small enough to complete and demonstrate inside the sprint; split it if not.

Definition of Done (for each story)

- Acceptance criteria pass, including negative cases and authorization failures.
- Server-side input validation and meaningful success/error/loading feedback are implemented.
- Automated tests are added for new backend routes; CI checks pass.
- Code is reviewed and merged through a pull request; no secrets or real credentials are committed.
- API behaviour and setup/configuration notes are documented; the feature is integrated into the deployed or shared test build.
- The team can demonstrate the story and record any known limitation in the backlog.

Suggested coordination for a 10-person team

- Create small feature squads (for example, 2–3 people per module) but appoint one integration owner to review API contracts and resolve cross-module conflicts.
- Assign clear ownership for frontend, backend/data, QA/CI and deployment, while rotating code review so every module has a second reviewer.
- Avoid ten people independently editing the same core models. Agree User, Project, Membership, Task and ActivityEvent schemas before parallel implementation.
- Keep a single sprint board and require each story to have an owner, reviewer, test evidence and pull request.
- Reserve capacity each sprint for integration, regression tests, bug fixes and demo preparation rather than planning every person at 100% feature work.

EPIC / priority verification and decisions to resolve

The EPIC breakdown is structurally consistent: it lists 13 EPICs, assigns US-01 through US-58 exactly once, and preserves the priorities in the revised user-story document. However, the merged file also contains a corrected FR/NFR document whose priorities or behaviour conflict with some story text. These are source inconsistencies, not silently resolved changes. Agree the source of truth with the team/instructor before implementing the affected stories.

Item	Conflict between uploaded documents	Decision needed
US-19 Direct messages	User Stories: Must; corrected FR-6: Should	Agree a single source of truth before sprint commitment.
US-25 Create milestone	User Stories: Must; corrected FR-7c: Should	Confirm priority; FR-7c groups tasks under milestones and is marked Should.
US-27/US-28 Kanban	User Stories: Must; corrected FR-8: Should	Confirm whether Kanban is Must or Should.
US-30 Notifications	User Stories: Must; corrected FR-10: Should	Confirm priority and ensure notification timing remains testable.
US-35/US-36 Decisions	User Stories: Must; corrected FR-13/FR-34: Should	Confirm priority before estimating.
US-38 Documentation	User Stories: Should; corrected FR-14b: Could	Confirm whether documentation is in the demo's required scope.
US-49 Portfolio	User Stories: Should; corrected FR-21: Could	Confirm scope; it affects discovery/portfolio dependency planning.
US-54 Follow users	User Stories: Must; corrected FR-26: Should	Confirm priority.
US-55/US-56 Moderation	User Stories: US-55 Must; corrected FR-27: Should	Align story priority with FR-27 and define moderation action outcomes.
US-02 Login session	US-02 story says JWT expires after 24 hours; corrected FR-1/NFR-3b says access token 15 minutes + revocable refresh token 7 days	Resolve before implementation; security requirements conflict.
Task statuses / blocked work	US-24 describes To Do, In Progress, Done; corrected FR-7b includes Blocked; US-40 calls blocked a label	Choose one model: Blocked status or label, and update acceptance tests.
Votes/profile totals	US-52 says one vote once; corrected FR-24 permits changing/retracting a vote. US-54 says total likes/upvotes; corrected FR-26 defines total vote score as sum of net post scores	Align behaviour and naming (vote/score, not likes) before social-feed implementation.
Project status	US-15 describes Active/Completed and reopen; corrected FR-3 adds Archived	Confirm whether Archived is in scope and define who can set it.

Scope discipline: the revised user-story document labels some items “New (review)” and says the team must confirm whether they are in scope. Do not assume the EPIC grouping approves those items. Keep unconfirmed items out of a committed sprint or mark them explicitly as conditional.

Suggested order of decisions: (1) session/token design, (2) task Blocked status versus label, (3) priorities for Kanban/milestones/notifications/DMs/decisions/documentation/portfolio/following/moderation, (4) vote change/retract semantics and profile score definition, (5) Archived project status, (6) which “New (review)” stories are in scope.